VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

Design Phase (Pt.1)

Software Engineering

KIEU Quoc Viet HUYNH Vinh Nam

ICT Department, USTH

Agenda

1. Review

Review

Requirements Analysis

What are we building?

Functional & non-functional specs.

Design

How are we going to build it?

System architecture & component design.

Development (Implementation)

Let's build it!

Writing code based on the design.

Testing (Verification)

Does it work correctly?

Testing & validating the system.

Deployment

Ship it to the world!

Releasing the product to production.

Maintenance

Keep it alive!

Updates, patches & ongoing support.

2. Design Architecture

Software Architecture Introduction

Architecture = the **highest level** of design — the least specific, most structural decisions.

Analogy: Designing a City

Before building anything, you **zone** the areas: residential here, commercial there, industry over there.

No details yet — just the **big picture structure**.

In Software

We take the **requirements document** and build a plan for *how* the system should function — before a single line of code is written.

Software Architecture Introduction

Architecture = the **highest level** of design — the least specific, most structural decisions.

Analogy: Designing a City

Before building anything, you **zone** the areas: residential here, commercial there, industry over there.

No details yet — just the **big picture structure**.

In Software

We take the **requirements document** and build a plan for *how* the system should function — before a single line of code is written.

Architects = the link between *idea* and *reality*

Not the ones who had the idea. Not the ones who build it.

The ones who create the **blueprint** so others can build it.

Why Architecture Matters

Bad architecture cannot be fixed with good programming.

Why Architecture Matters

Bad architecture cannot be fixed with good programming.

Analogy: Skyscraper Foundation

If you find a **foundation problem** at 20 storeys high:

- Fixing it is enormously harder than at ground level
- You may have to **tear the whole thing down**
- Fast to fix early — catastrophic to fix late

Why Architecture Matters

Bad architecture cannot be fixed with good programming.

Analogy: Skyscraper Foundation

If you find a **foundation problem** at 20 storeys high:

- Fixing it is enormously harder than at ground level
- You may have to **tear the whole thing down**
- Fast to fix early — catastrophic to fix late

In Software

Poor architecture leads to:

- Products that take too long to **maintain**
- Very hard to **extend** over time
- Often the only fix is to **scrap and restart**
- Or worse — **abandon** the software entirely

Why Architecture Matters

Bad architecture cannot be fixed with good programming.

Analogy: Skyscraper Foundation

If you find a **foundation problem** at 20 storeys high:

- Fixing it is enormously harder than at ground level
- You may have to **tear the whole thing down**
- Fast to fix early — catastrophic to fix late

In Software

Poor architecture leads to:

- Products that take too long to **maintain**
- Very hard to **extend** over time
- Often the only fix is to **scrap and restart**
- Or worse — **abandon** the software entirely

Once a poorly architected system is deployed, no amount of good programmers can save it — the damage is already built in.

Bad Architecture in Practice

Scenario: Tightly Coupled Components

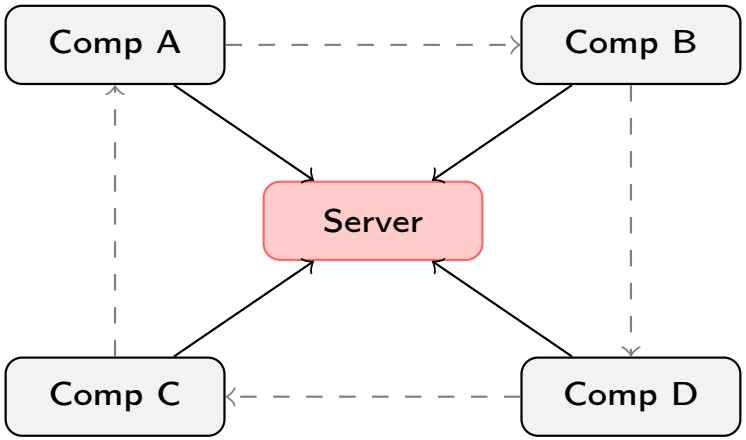
Every component in the app connects **directly** to a central server, with many crosslinks between components.

One day: the central server has a **security flaw**.

To fix it, you must **replace the server** and update every single connection — 3, 4, or more endpoints per component.

Result:

- Cannot hot-swap the server
- Must reprogram the **entire app**
- Choose: stay insecure, rebuild, or abandon



Replace server → reprogram everything

Software Architecture Overview

Software architecture = breaking larger systems into **smaller, focused components**.

Benefits of Good Architecture

Faster development — multiple engineers work in parallel

Less idle time — no waiting for others to finish

Easier debugging — breaks are easy to locate

Buy or build — cleanly swap in third-party components

Lower long-term cost — easier to maintain and extend

Software Architecture Overview

Software architecture = breaking larger systems into **smaller, focused components**.

Benefits of Good Architecture

- **Faster development** — multiple engineers work in parallel
- **Less idle time** — no waiting for others to finish
- **Easier debugging** — breaks are easy to locate
- **Buy or build** — cleanly swap in third-party components
- **Lower long-term cost** — easier to maintain and extend

Without Architecture: Spaghetti Code

- Files link to each other randomly
- Hard to trace bugs across many calls
- Engineers block each other → merge conflicts
- Changing one thing breaks everything else
- No clear owner for any part of the code

Software Architecture Overview

Software architecture = breaking larger systems into **smaller, focused components**.

Benefits of Good Architecture

- **Faster development** — multiple engineers work in parallel
- **Less idle time** — no waiting for others to finish
- **Easier debugging** — breaks are easy to locate
- **Buy or build** — cleanly swap in third-party components
- **Lower long-term cost** — easier to maintain and extend

Without Architecture: Spaghetti Code

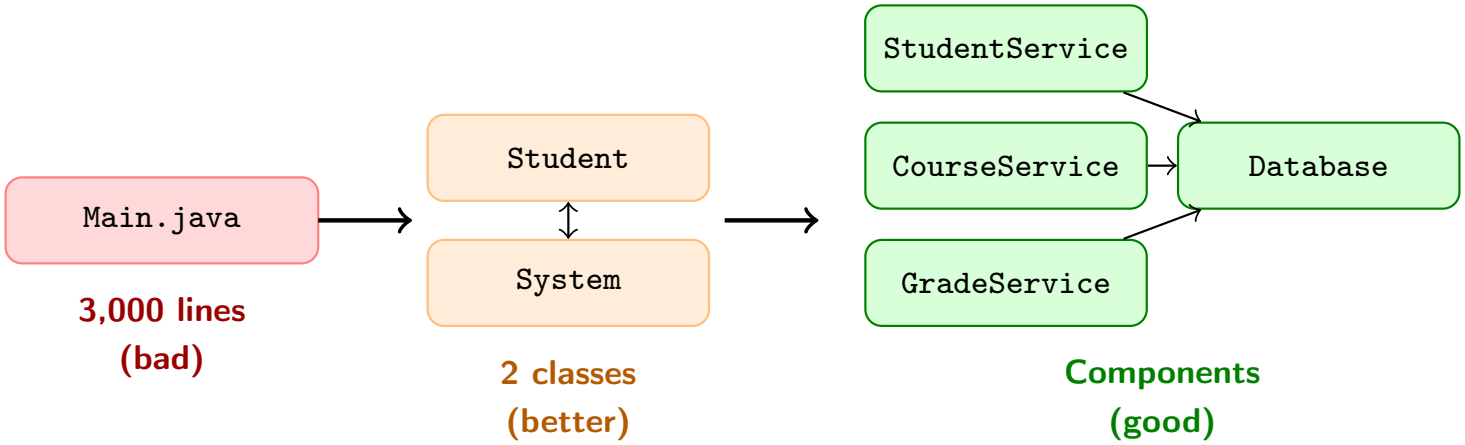
- Files link to each other randomly
- Hard to trace bugs across many calls
- Engineers block each other → merge conflicts
- Changing one thing breaks everything else
- No clear owner for any part of the code

Architecture mistakes are almost impossible to fix once coding has begun.

Fixing bad architecture costs more time and money than starting over from scratch.

Breaking Down Architecture — Example

Starting from a **single class** → gradually breaking into focused components:



"God" Class (bad)

- Everything stuffed into `Main.java`
- 5 team members edit the same file → conflicts
- Bug in grading? Search 3,000 lines to find it

Separated Components (good)

- Grade bug? → open `GradeService`, read 150 lines, done
- Each teammate owns a clear class
- Swap `Database` without touching other services

Architecture at a Smaller Scale — You've Seen This Before

Good architecture = each component does **one thing**, and can be **replaced independently**.

Architecture at a Smaller Scale — You've Seen This Before

Good architecture = each component does **one thing**, and can be **replaced independently**.

From your ADS course:

You often write sort logic **directly inside** `main()` — switching from BubbleSort to QuickSort means **rewriting** the whole block.

```
// sort logic tangled into
main()
for (int i=0; i < n-1; i++) {
...  }
```

Architecture at a Smaller Scale — You've Seen This Before

Good architecture = each component does **one thing**, and can be **replaced independently**.

From your ADS course:

You often write sort logic **directly inside** `main()` — switching from BubbleSort to QuickSort means **rewriting** the whole block.

```
// sort logic tangled into
main()
for (int i=0; i < n-1; i++) {
...  }
```

A cleaner design:

Extract the algorithm into its own component. Swapping the strategy = **change one line**.

```
Sorter s = new QuickSorter();
s.sort(data);
// later:  swap to MergeSort
Sorter s = new MergeSorter();
```

Architecture at a Smaller Scale — You've Seen This Before

Good architecture = each component does **one thing**, and can be **replaced independently**.

From your ADS course:

You often write sort logic **directly inside** `main()` — switching from BubbleSort to QuickSort means **rewriting** the whole block.

```
// sort logic tangled into
main()
for (int i=0; i < n-1; i++) {
...  }
```

A cleaner design:

Extract the algorithm into its own component. Swapping the strategy = **change one line**.

```
Sorter s = new QuickSorter();
s.sort(data);
// later:  swap to MergeSort
Sorter s = new MergeSorter();
```

This is **architecture thinking** — not just at the system level, but at every level of design.
The same principle scales from a single class to an entire software system.

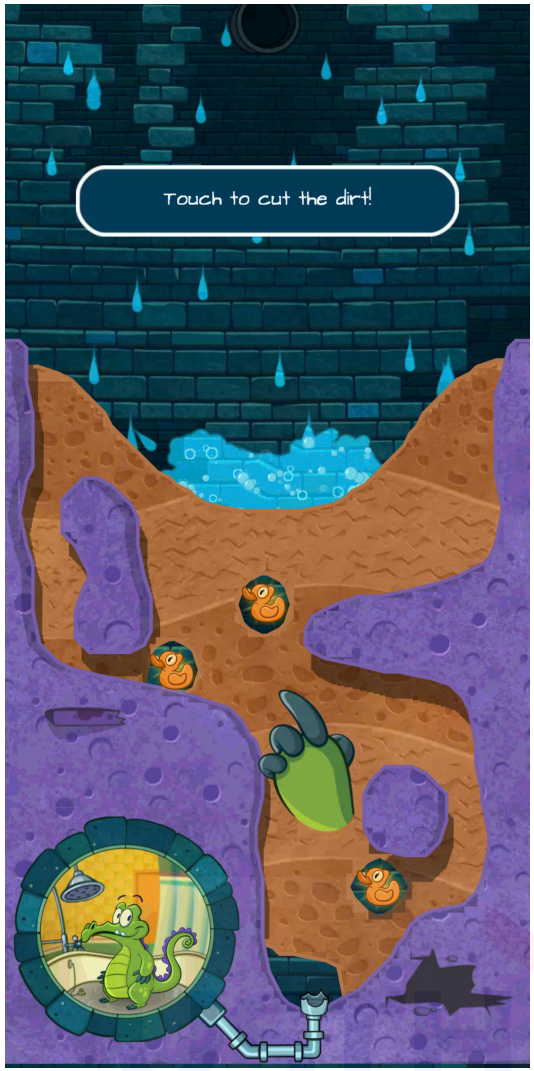
Software Architecture — A Practical Example

The App: Where's My Water?

- Player draws lines to carve through dirt
- Water flows through the carved path
- Collect rubber ducks along the way
- Deliver water to Swampy's shower to complete the level
- Score tracked by number of ducks collected (1–3 per level)

Right now - everything runs in *one single file*.

How do we break this into a proper architecture?



Architecture Level 1 — Three Layers

Front End

Graphics & Assets

Front End

Everything the player sees:
background, character, menus, UI

Architecture Level 1 — Three Layers

Front End

Graphics & Assets

Game Logic

Controls & State

Front End

Everything the player sees:
background, character, menus, UI

Game Logic

Water physics, duck collision,
level state, game loop

Architecture Level 1 — Three Layers

Front End

Graphics & Assets

Game Logic

Controls & State

Server

User Data & Scores

Front End

Everything the player sees:
background, character, menus, UI

Game Logic

Water physics, duck collision,
level state, game loop

Server

Stores user accounts, scores, level progression

Architecture Level 1 — Three Layers

Front End

Everything the player sees:
background, character, menus, UI

Game Logic

Water physics, duck collision,
level state, game loop

Server

Stores user accounts, scores, level progression

Architecture Level 1 — Three Layers

Front End never touches Server directly — always through Game Logic

Front End

Everything the player sees: background, character, menus, UI

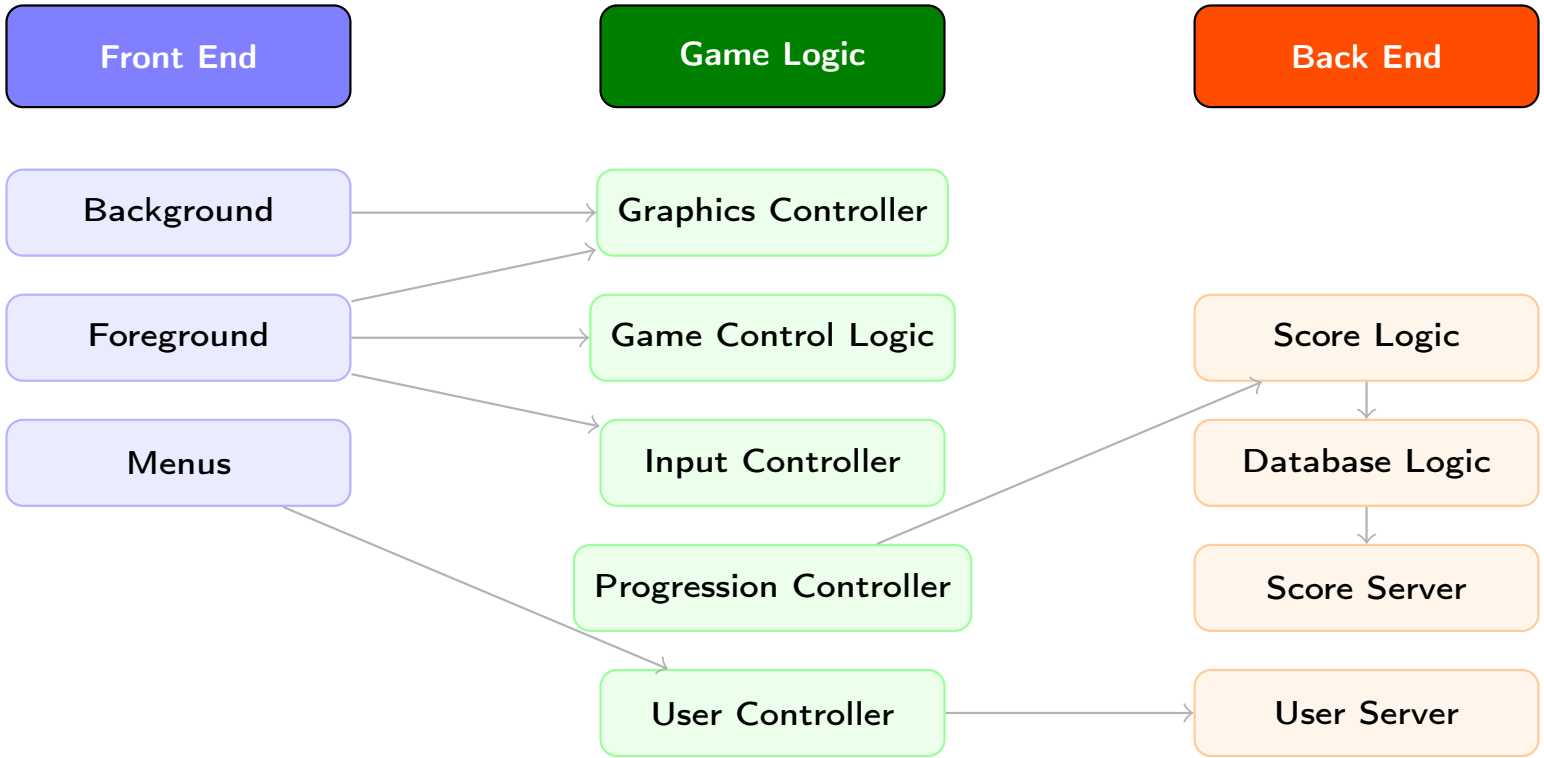
Game Logic

Water physics, duck collision, level state, game loop

Server

Stores user accounts, scores, level progression

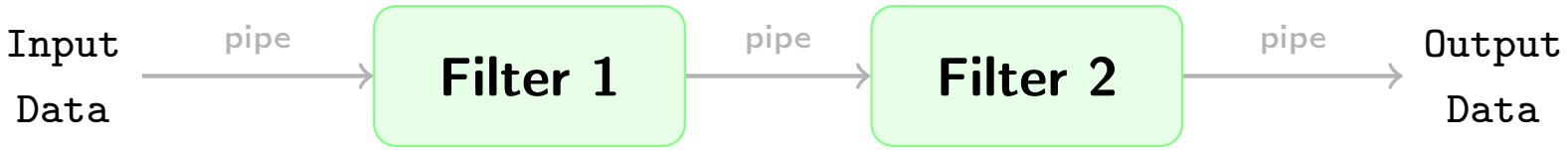
Architecture Level 2 — Drilling Down



Keep asking: Does this make sense? Can it be broken down further? Is everything in the right layer?

3. Design Patterns

Pipe and Filter Pattern



Pipe

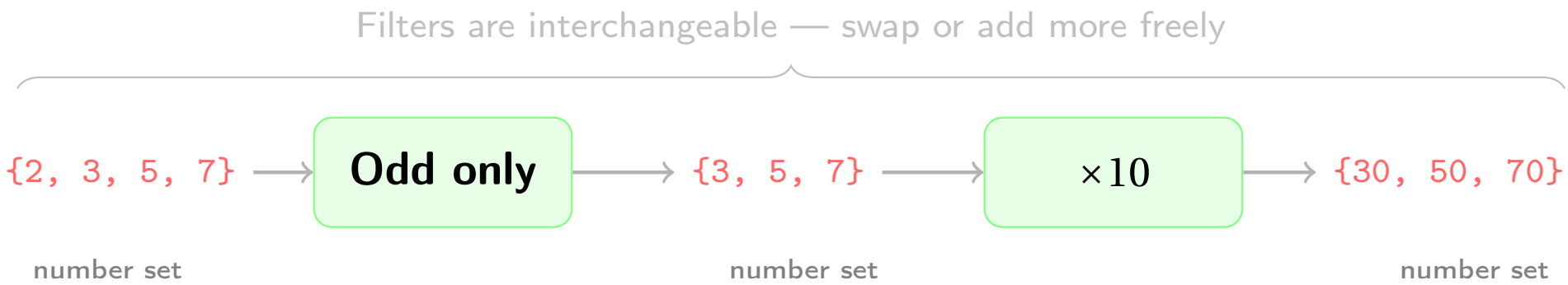
The **connection** between components — carries data from one filter to the next

Filter

A **processing unit** — receives data, transforms it, and passes it on

Patterns are a **toolbox** — combine them freely to fit your specific problem

The Key Rule: Type In = Type Out



★ The Rule

Same type in, same type out.

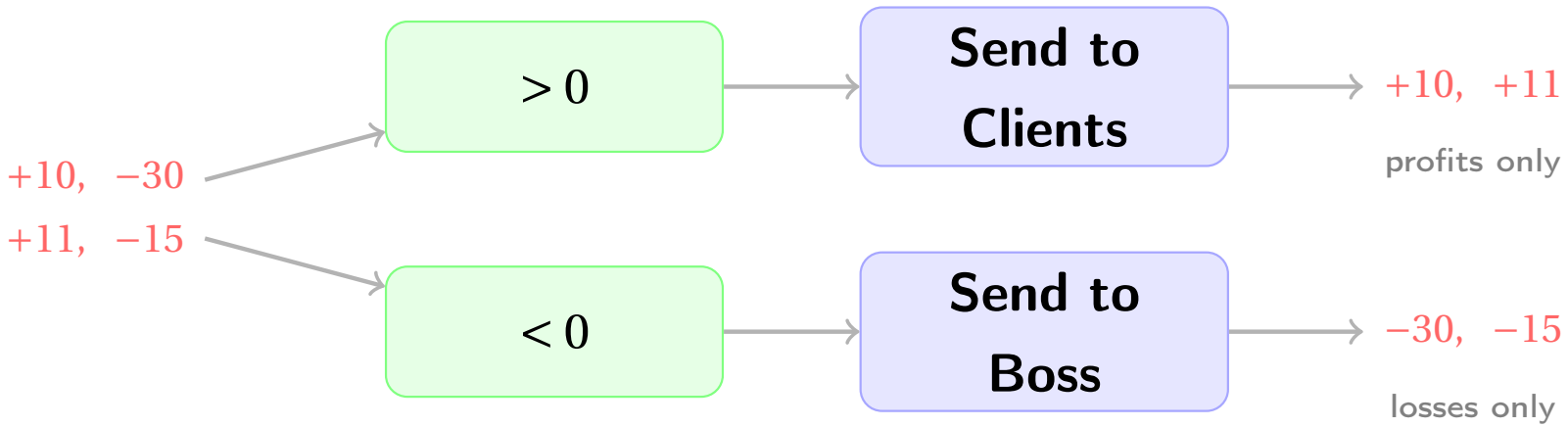
Size may change — type must not.

⇒ Filters become true black boxes

Actions

Do something with the data & pass it through unchanged

Example: Investment Data Pipeline



Clients receive only gains ⇒ confidence maintained

Boss receives only losses ⇒ can act to fix them

Filters branch freely — each chain stays independent and composable

Pipe and Filter in the Real World

Unix Shell Pipeline

```
cat log.txt | grep ERROR | sort
| uniq -c
```

Each command is a filter — output of one feeds directly into the next

Data ETL Pipeline

Raw data → Extract → Transform → Validate → Load to database

Core of data warehouses: Apache Kafka, Spark, Airflow

Image Processing

Raw image → Noise reduction → Edge detection → Color correction → Output

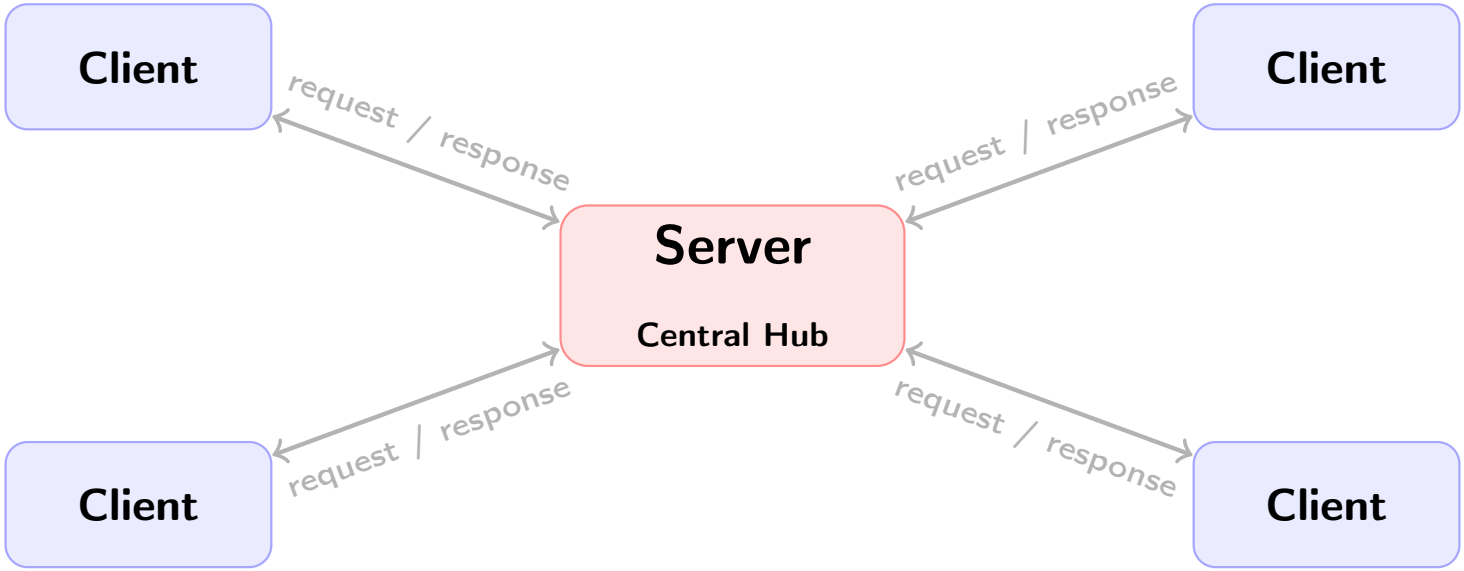
Used in camera apps, medical imaging, satellite photo processing

CI/CD Pipeline

Code commit → Build → Unit test → Integration test → Deploy

GitHub Actions, Jenkins, GitLab CI all follow this pattern

Client-Server Pattern



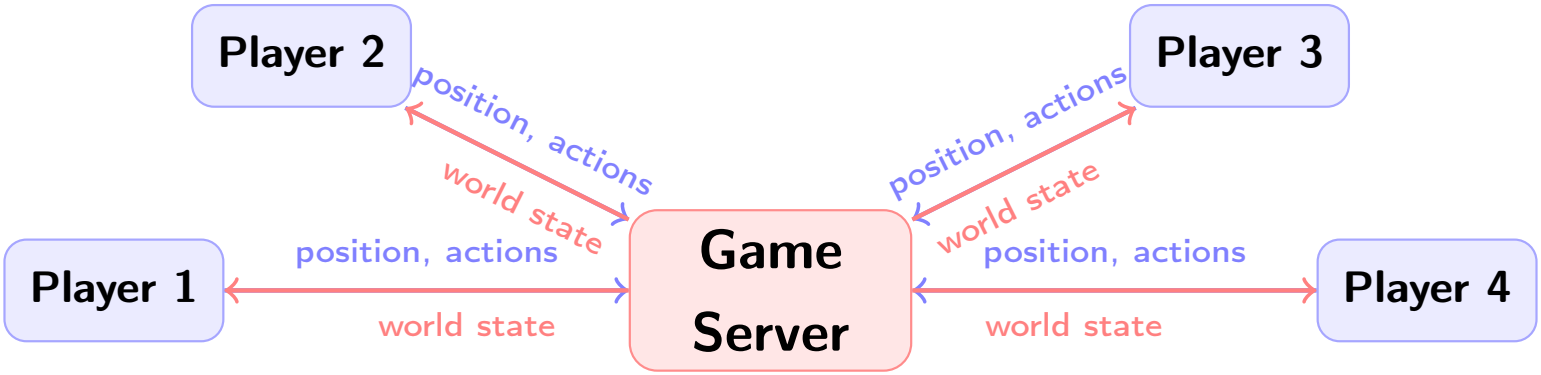
Server

Central hub of information — stores data, processes requests, distributes responses

Clients

Contact the server to request data — receive and display what the server sends back

Example: Online Gaming Server



Client sends (every few ms)

- Position
- Speed
- Items held
- Actions taken

Server broadcasts back

- All players' positions
- Collision results
- Updated world state

★ Server Lag

Response arrives late ⇒ client renders player at wrong position ⇒ character teleports across screen

Client Software & Server Software

Server Software

- Runs on the central machine
- Stores and manages all shared data
- Processes incoming requests
- Sends responses back to clients

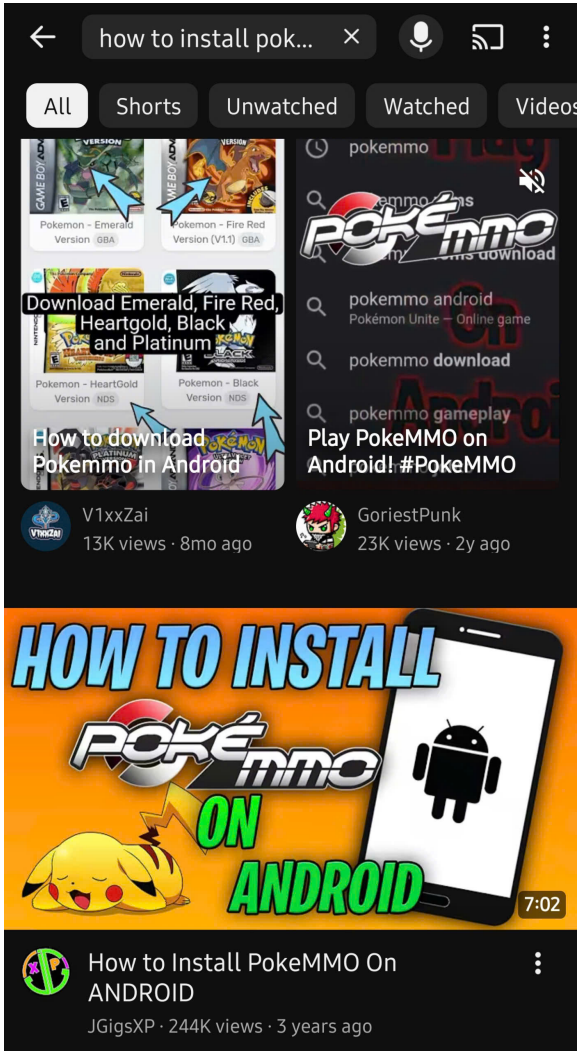
Client Software

- Runs on each user's device
- Sends requests to the server
- Receives and displays data
- Never talks directly to other clients

This is how the entire internet works

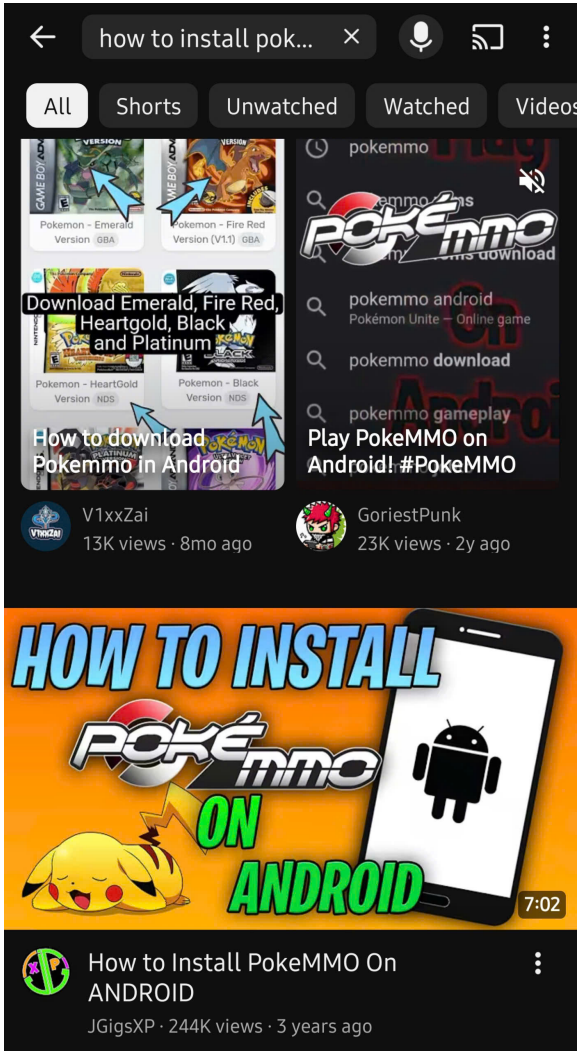
- Visit `facebook.com` → your browser (client) contacts Facebook's server
- Server returns HTML, images, timeline data
- Every new page = a new request → a new response

Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

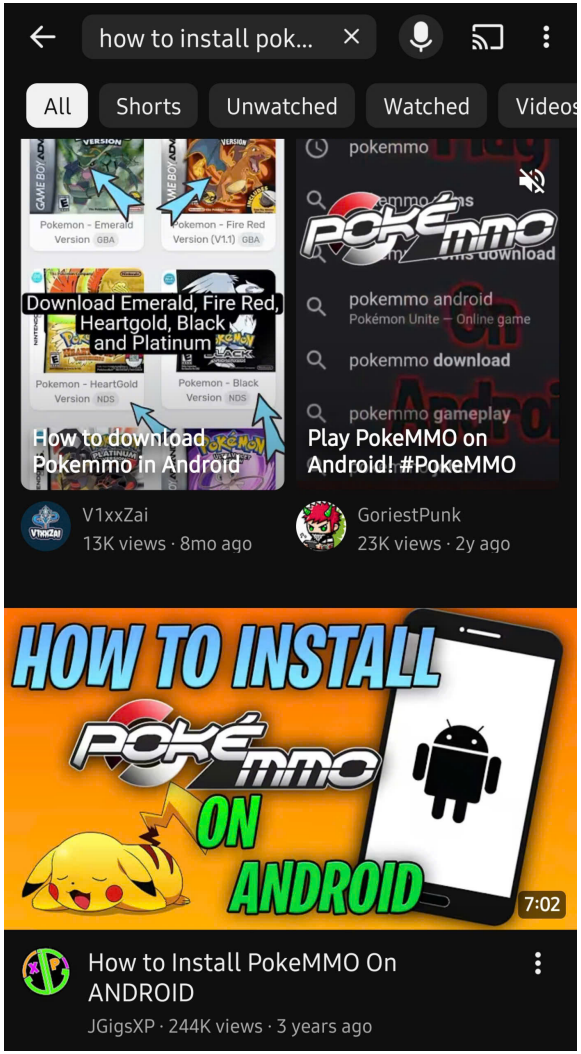
Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

Open app → server sends recommended videos

Client-Server in the Real World — YouTube

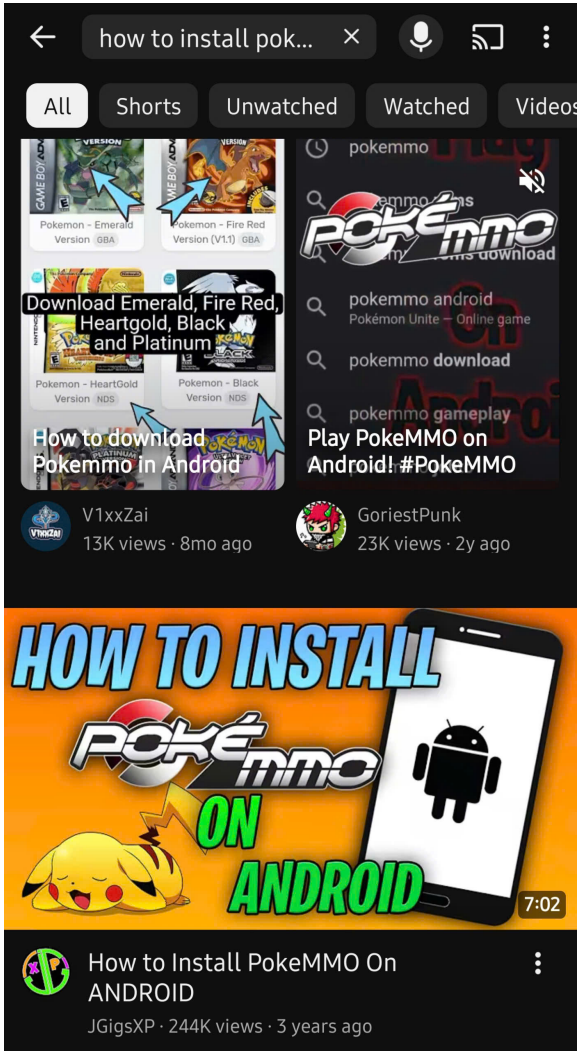


Every tap is a request to YouTube's server

Open app → server sends recommended videos

Press play → server streams video in chunks

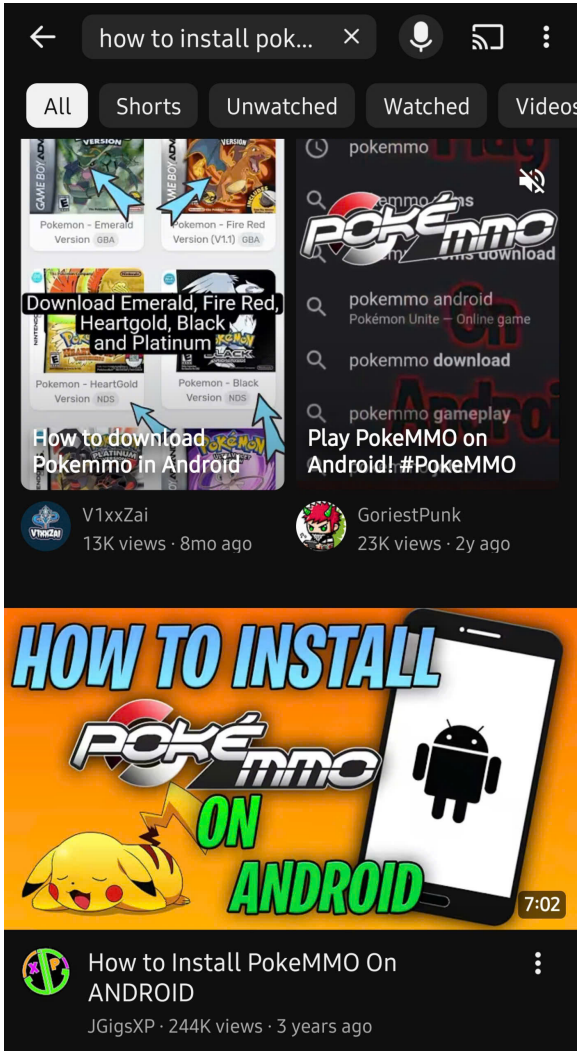
Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

- Open app → server sends recommended videos
- Press play → server streams video in chunks
- Scroll comments → server loads more data

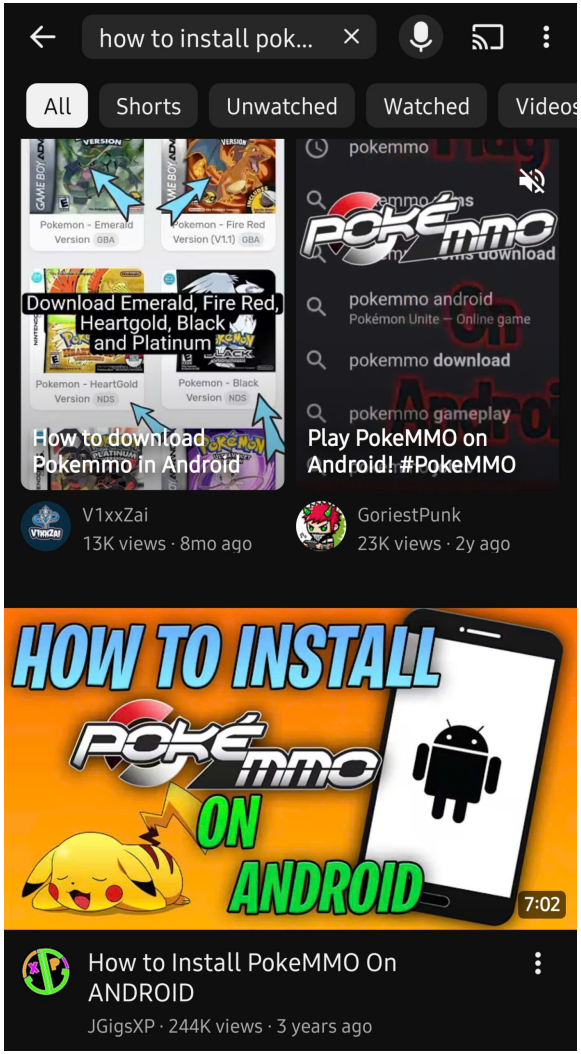
Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

- Open app → server sends recommended videos
- Press play → server streams video in chunks
- Scroll comments → server loads more data
- Like a video → client tells server to update the count

Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

Open app → server sends recommended videos

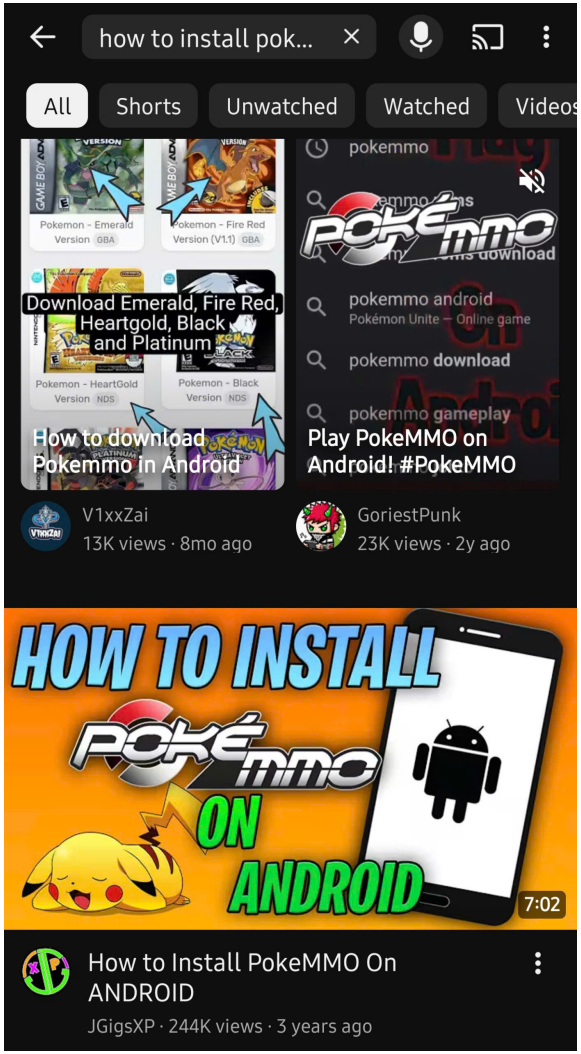
Press play → server streams video in chunks

Scroll comments → server loads more data

Like a video → client tells server to update the count

Without the server. . .

Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

Open app → server sends recommended videos

Press play → server streams video in chunks

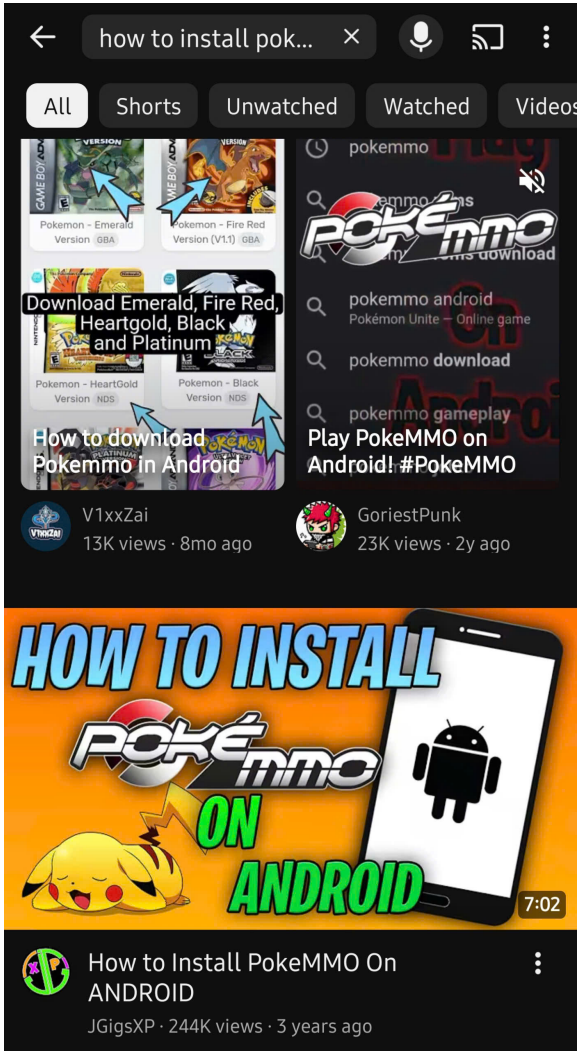
Scroll comments → server loads more data

Like a video → client tells server to update the count

Without the server...

No videos load. No search works.

Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

Open app → server sends recommended videos

Press play → server streams video in chunks

Scroll comments → server loads more data

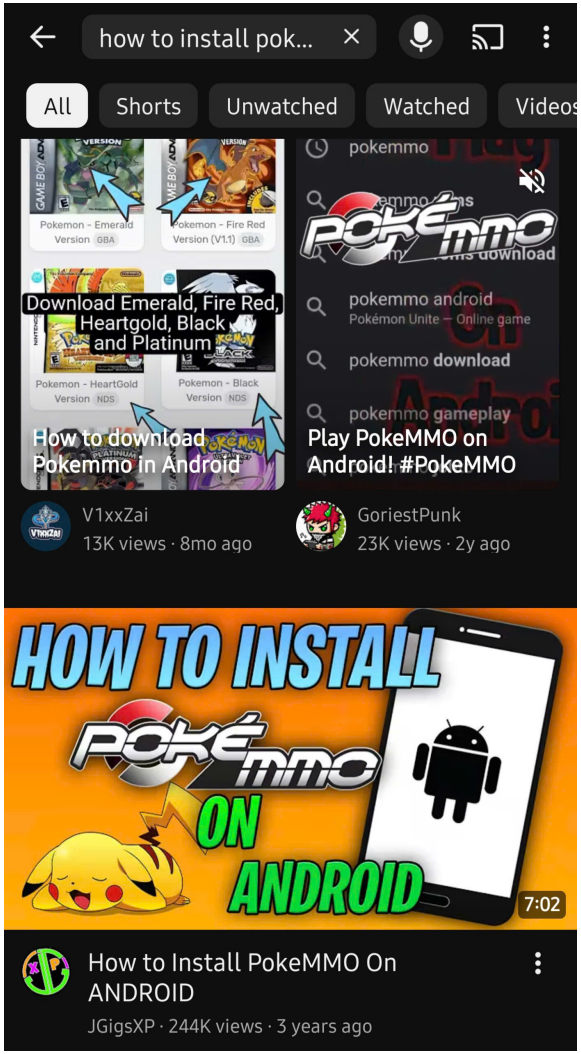
Like a video → client tells server to update the count

Without the server...

No videos load. No search works.

The app is just an empty shell.

Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

Open app → server sends recommended videos

Press play → server streams video in chunks

Scroll comments → server loads more data

Like a video → client tells server to update the count

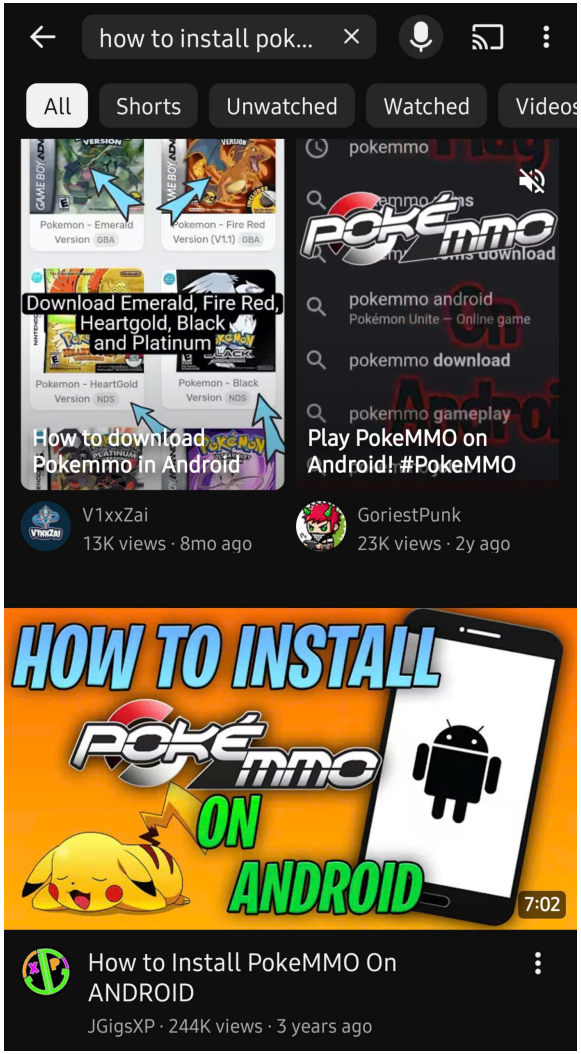
Without the server. . .

No videos load. No search works.

The app is just an empty shell.

⇒ The client **displays**. The server **owns the data**.

Client-Server in the Real World — YouTube



Every tap is a request to YouTube's server

Open app → server sends recommended videos

Press play → server streams video in chunks

Scroll comments → server loads more data

Like a video → client tells server to update the count

Without the server...

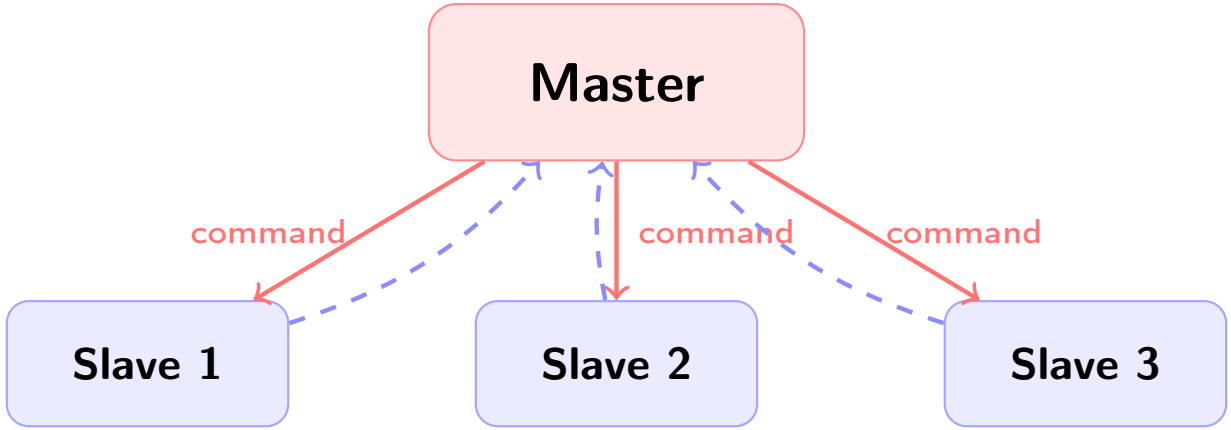
No videos load. No search works.

The app is just an empty shell.

⇒ The client **displays**. The server **owns the data**.

YouTube serves **500+ hours** of video uploaded every minute to **billions of clients** — all through this same pattern

Master-Slave Pattern



Slaves never communicate with each other

Master

Always at the top — issues commands, stays most up-to-date, controls all slaves

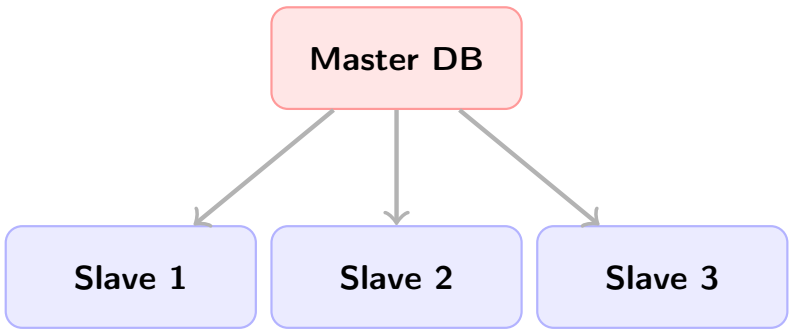
Slaves

Wait for commands — execute tasks, report back, never control the master

Master-Slave — Two Examples

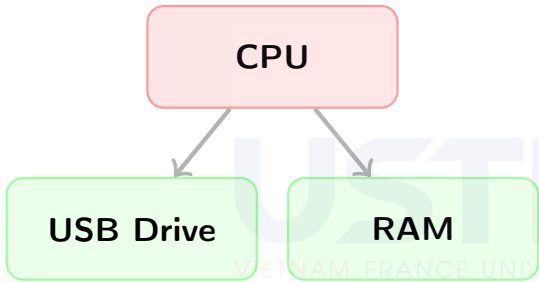
Database Replication

- Master DB: always live, accepts writes
- At midnight — master commands slaves to sync
- Slaves copy all new changes from master
- Go quiet again until next command



CPU & USB Drive

- CPU (master) plugs in USB drive (slave)
- CPU tells USB: *"copy your data to hard drive"*
- USB executes — CPU continues other tasks
- USB never issues commands back to CPU



Why Use Master-Slave?

✓ With Master-Slave

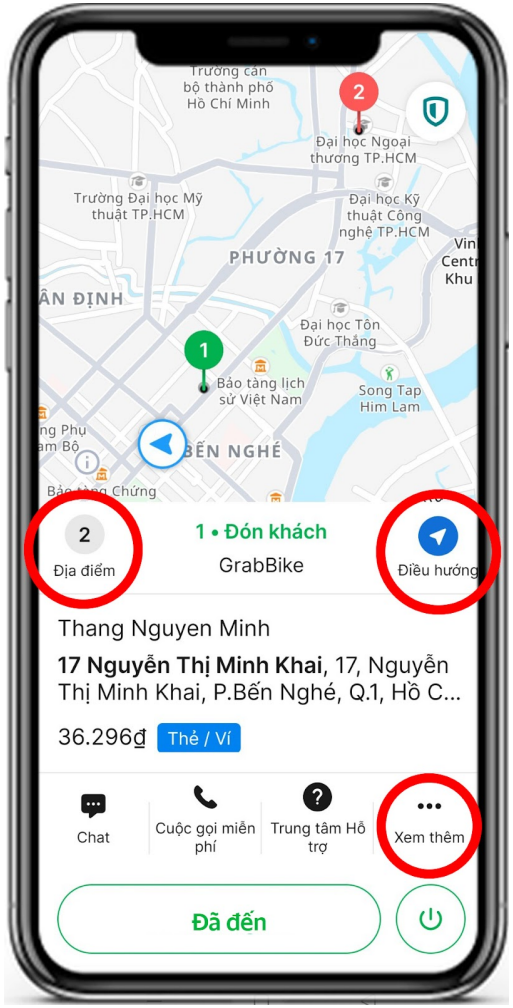
- Slaves are fully isolated from each other
- Swap or replace any slave freely
- Bad data? — must come from master
- Easy to trace errors up the chain

× Without It — Criss-Cross

- Every module talks to every other
- Information flow impossible to track
- One bug cascades everywhere
- Looks like spaghetti — hard to fix

A **Controller** is the software term for the master —
it gives orders down, receives messages up, and coordinates everything

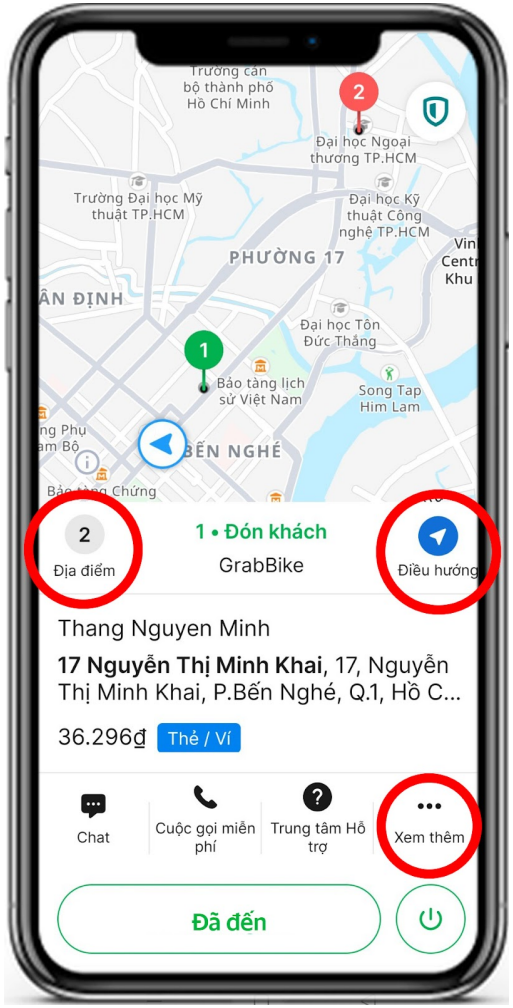
Master-Slave in the Real World — Grab



Master

Slave

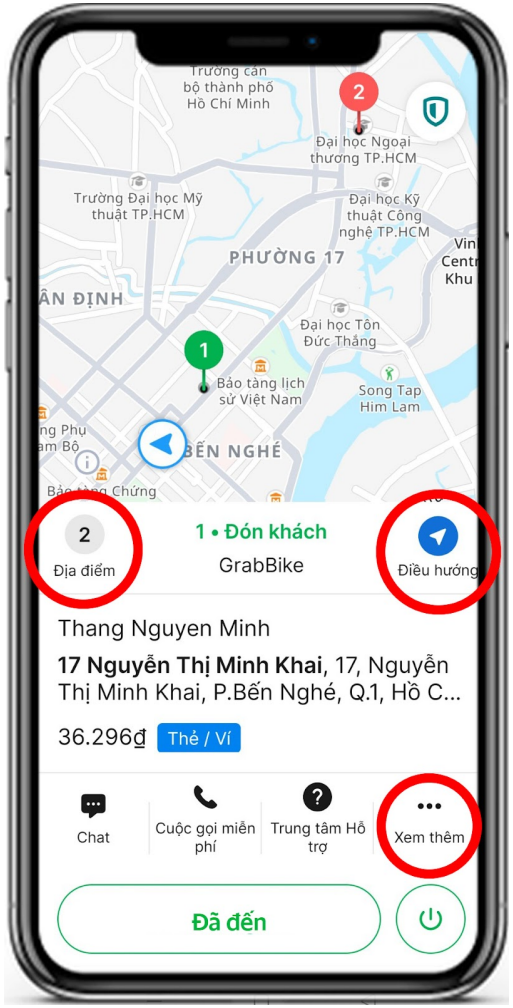
Master-Slave in the Real World — Grab



Master — Grab Central Server

Slave

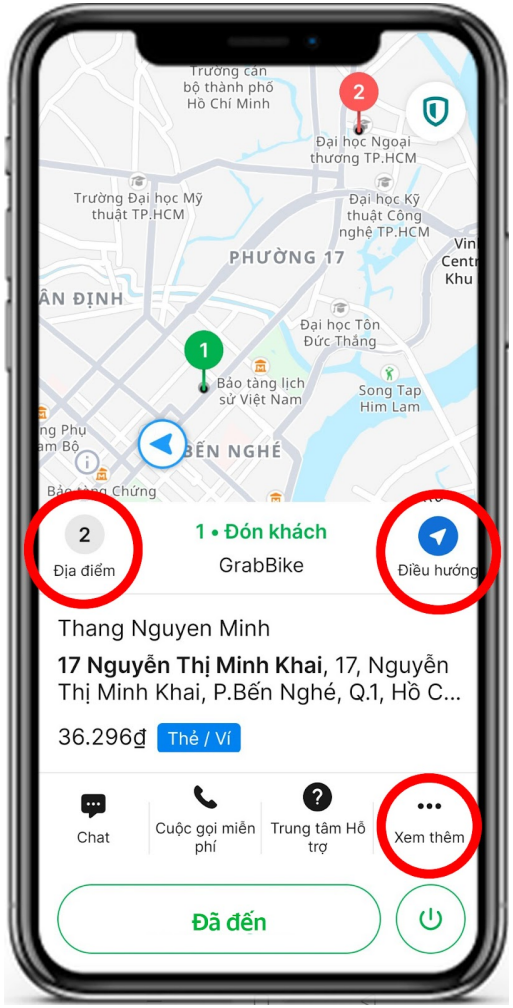
Master-Slave in the Real World — Grab



Master — Grab Central Server

Slave — Driver's App

Master-Slave in the Real World — Grab

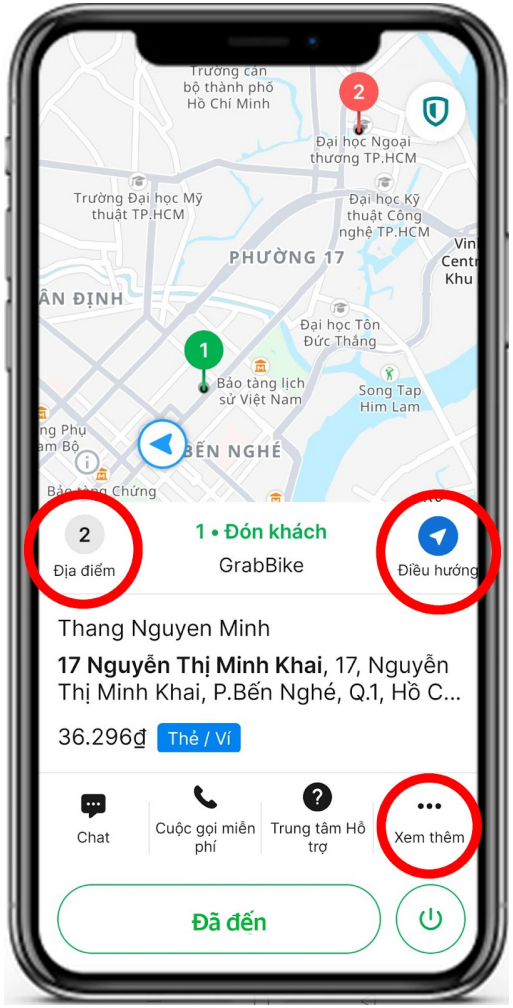


Master — Grab Central Server

Tracks all drivers in real time

Slave — Driver's App

Master-Slave in the Real World — Grab



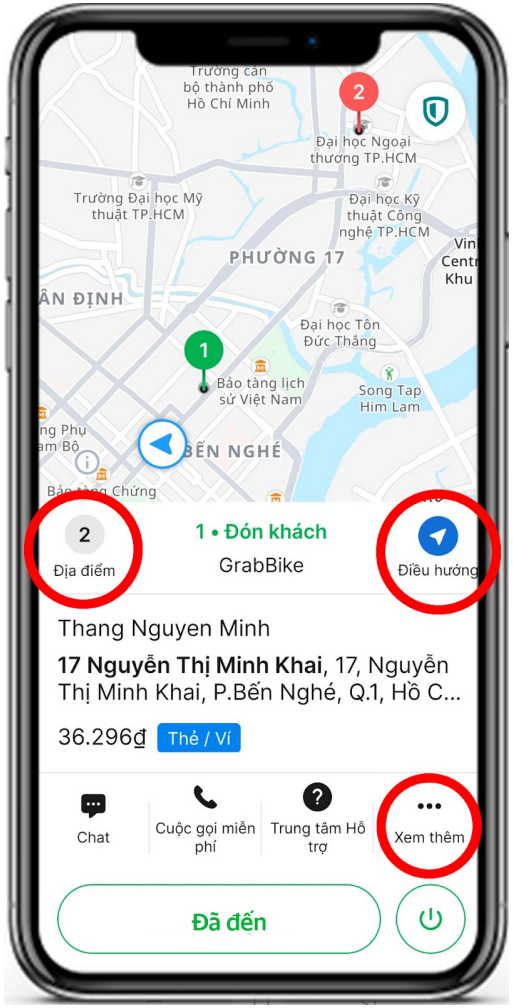
Master — Grab Central Server

Tracks all drivers in real time

Matches riders to nearest driver

Slave — Driver's App

Master-Slave in the Real World — Grab

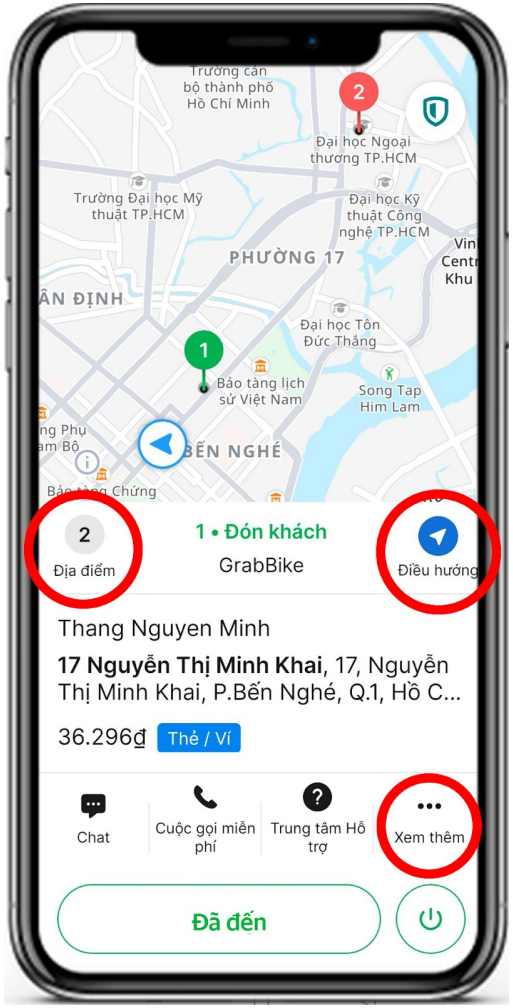


Master — Grab Central Server

- Tracks all drivers in real time
- Matches riders to nearest driver
- Sends commands: pick up, reroute, cancel

Slave — Driver's App

Master-Slave in the Real World — Grab

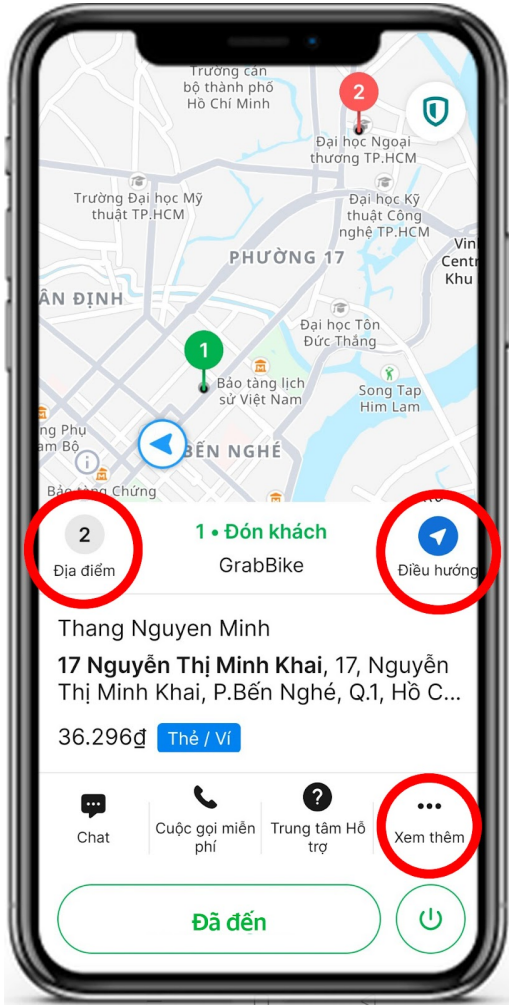


Master — Grab Central Server

- Tracks all drivers in real time
- Matches riders to nearest driver
- Sends commands: pick up, reroute, cancel
- Calculates fare and payment

Slave — Driver's App

Master-Slave in the Real World — Grab



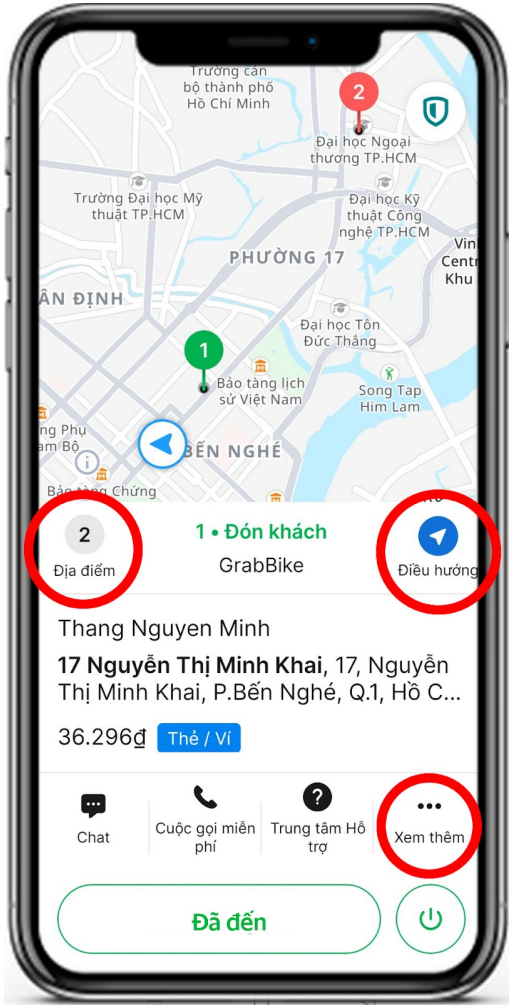
Master — Grab Central Server

- Tracks all drivers in real time
- Matches riders to nearest driver
- Sends commands: pick up, reroute, cancel
- Calculates fare and payment

Slave — Driver's App

- Sends location to server every few seconds

Master-Slave in the Real World — Grab



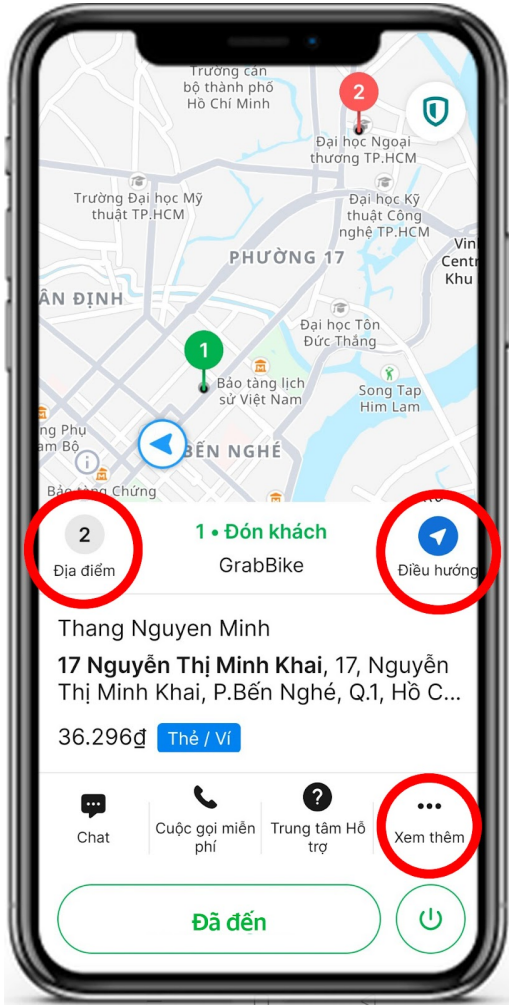
Master — Grab Central Server

- Tracks all drivers in real time
- Matches riders to nearest driver
- Sends commands: pick up, reroute, cancel
- Calculates fare and payment

Slave — Driver's App

- Sends location to server every few seconds
- Waits — does nothing until server sends a job

Master-Slave in the Real World — Grab



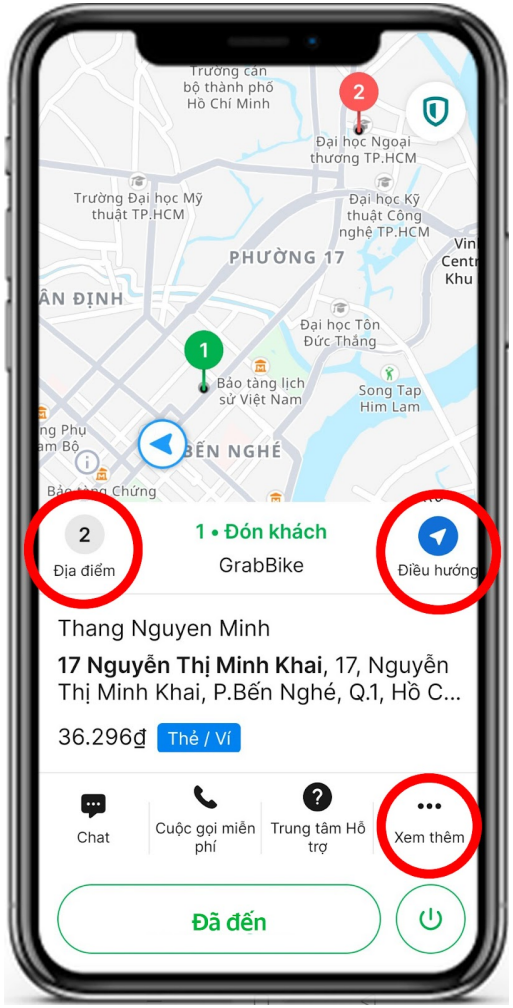
Master — Grab Central Server

- Tracks all drivers in real time
- Matches riders to nearest driver
- Sends commands: pick up, reroute, cancel
- Calculates fare and payment

Slave — Driver's App

- Sends location to server every few seconds
- Waits — does nothing until server sends a job
- Executes whatever the server assigns

Master-Slave in the Real World — Grab



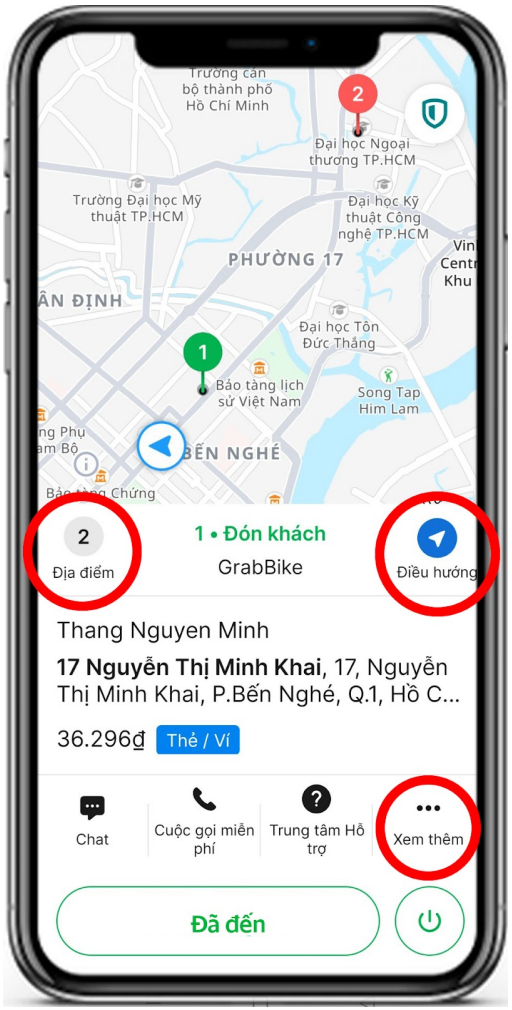
Master — Grab Central Server

- Tracks all drivers in real time
- Matches riders to nearest driver
- Sends commands: pick up, reroute, cancel
- Calculates fare and payment

Slave — Driver's App

- Sends location to server every few seconds
- Waits — does nothing until server sends a job
- Executes whatever the server assigns
- Never contacts other drivers directly

Master-Slave in the Real World — Grab



Master — Grab Central Server

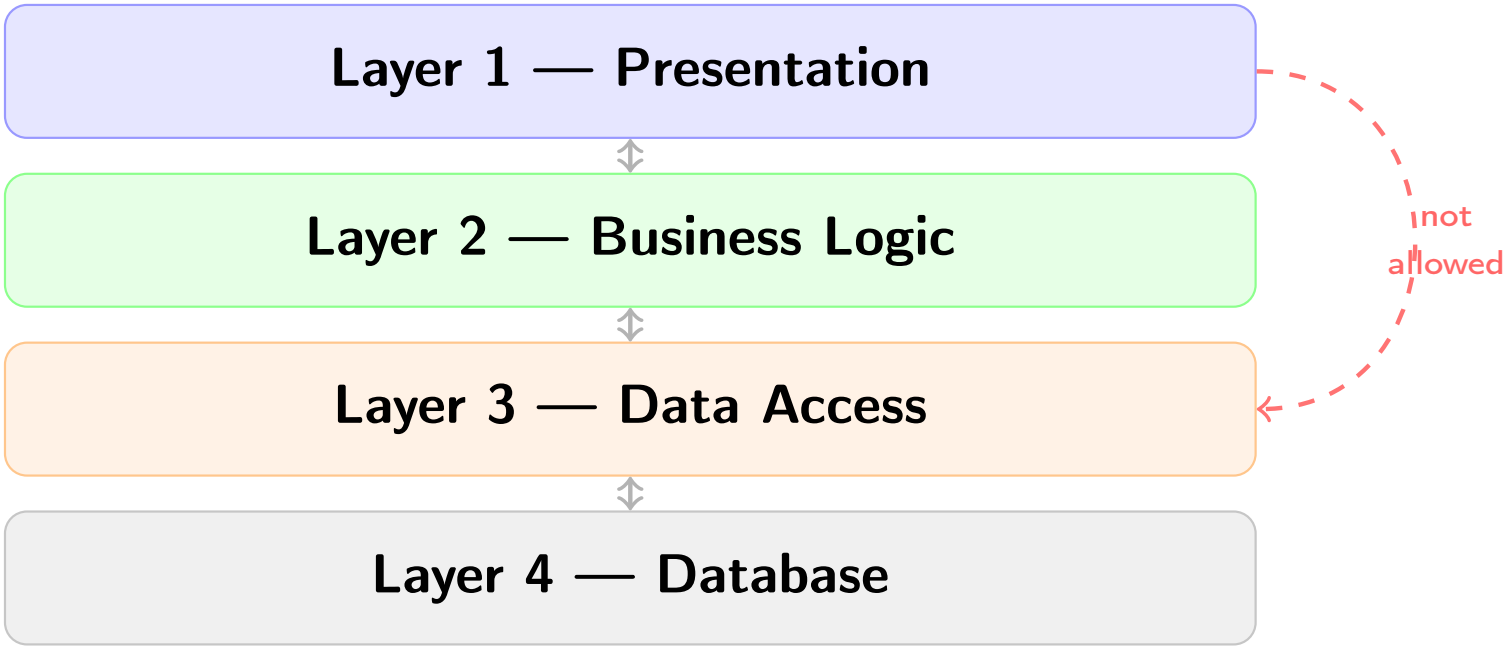
- Tracks all drivers in real time
- Matches riders to nearest driver
- Sends commands: pick up, reroute, cancel
- Calculates fare and payment

Slave — Driver's App

- Sends location to server every few seconds
- Waits — does nothing until server sends a job
- Executes whatever the server assigns
- Never contacts other drivers directly

Thousands of driver apps (slaves) -
All controlled by one central server (master)

Layered Pattern



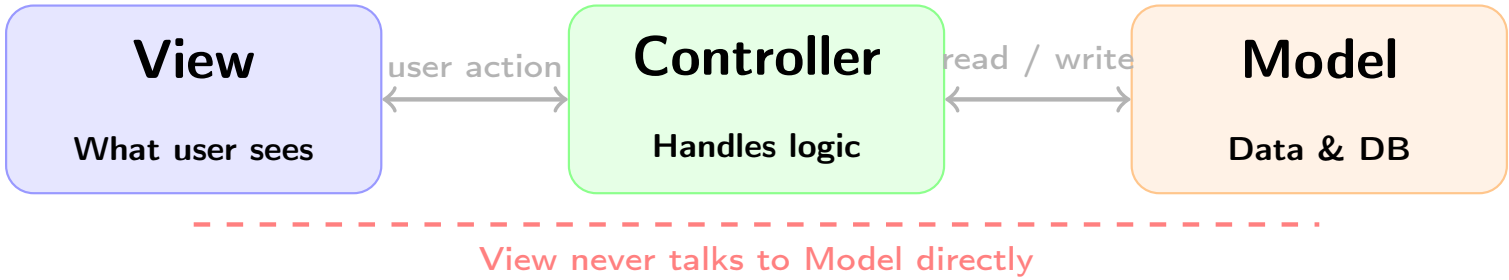
★ The Rule

Each layer only communicates with its **adjacent** layer — no skipping

The Trade-off

Data must transmit through each layer — a few extra steps, but much easier to debug

A Famous Example — Model View Controller (MVC)



View

Profile page — name field, email field, submit button

HTML + JavaScript

Controller

Receives submit event

Calls `updateUser()`

Passes data to Model

Model

Database row:

`username | email`

Updates on command

Why Use the Layered Pattern?

✓ Easier Debugging

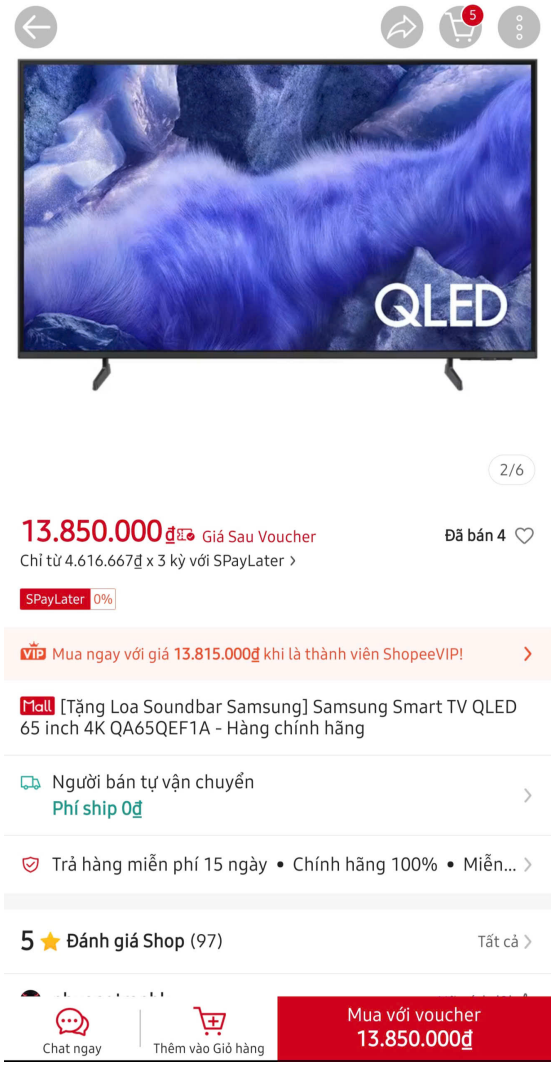
- UI broken? → check View layer
- Data not saving? → check Controller
- DB corrupt? → check Model layer
- Each bug has a **known address**

✓ Team Specialization

- Front-end devs own the View
- Back-end devs own Model + Controller
- Teams work in parallel without conflicts
- Swap out a layer without breaking others

Layered pattern = **classify, isolate, maintain** —
the antidote to spaghetti code

Layered Pattern in the Real World — Shopee



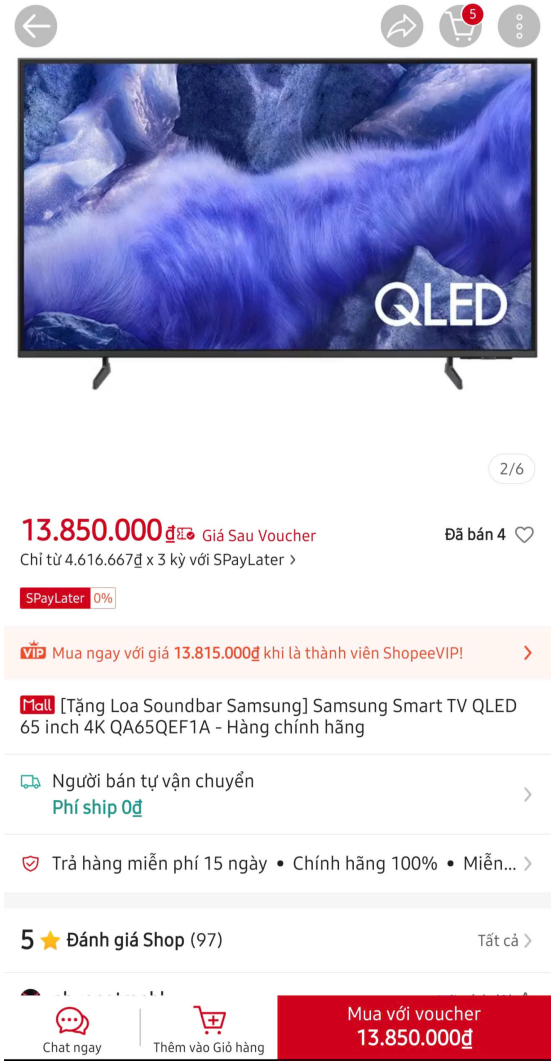
Presentation

Business Logic

Data Access

Database

Layered Pattern in the Real World — Shopee



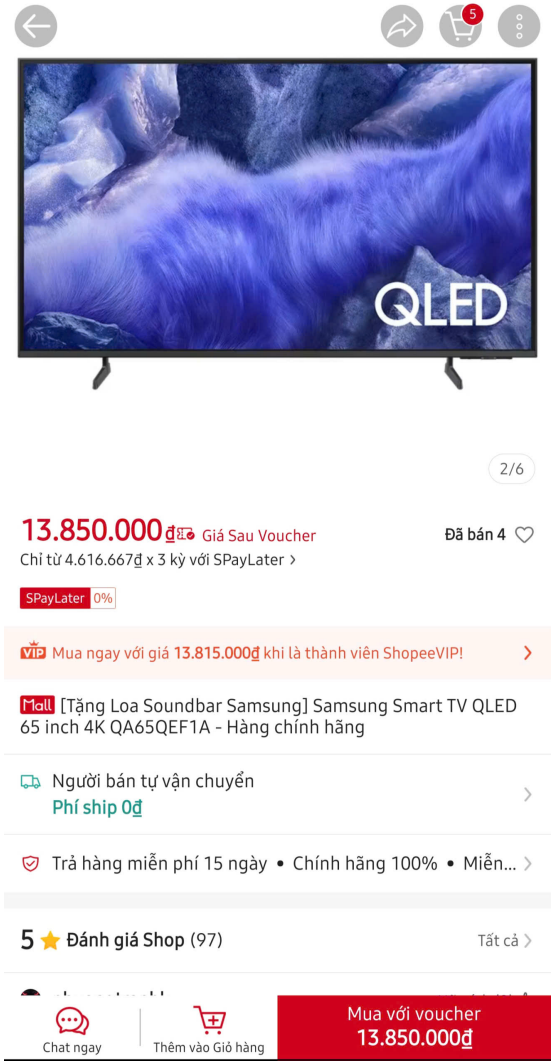
Presentation — UI, product listing, cart

Business Logic

Data Access

Database

Layered Pattern in the Real World — Shopee



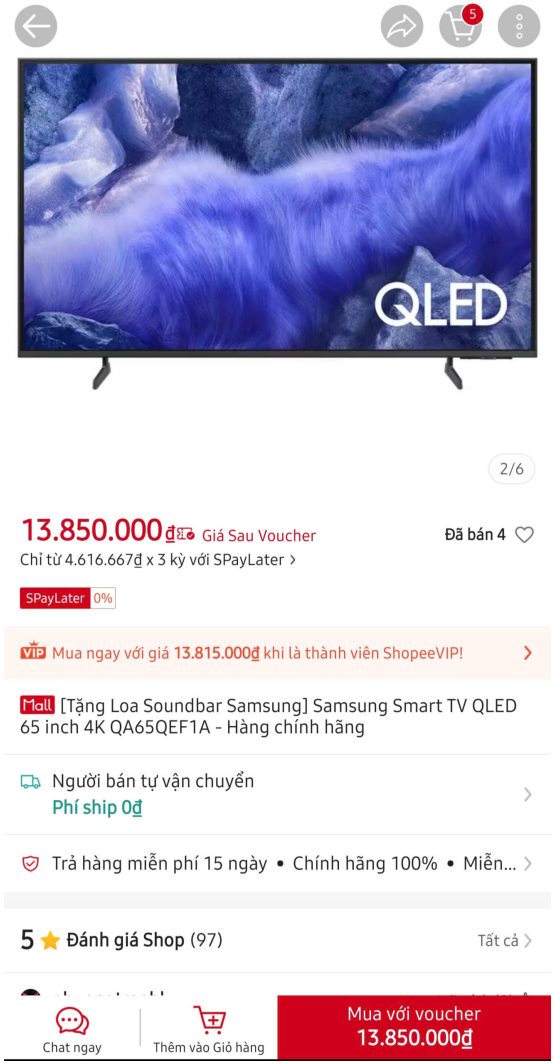
Presentation — UI, product listing, cart

Business Logic — pricing, vouchers, orders

Data Access

Database

Layered Pattern in the Real World — Shopee



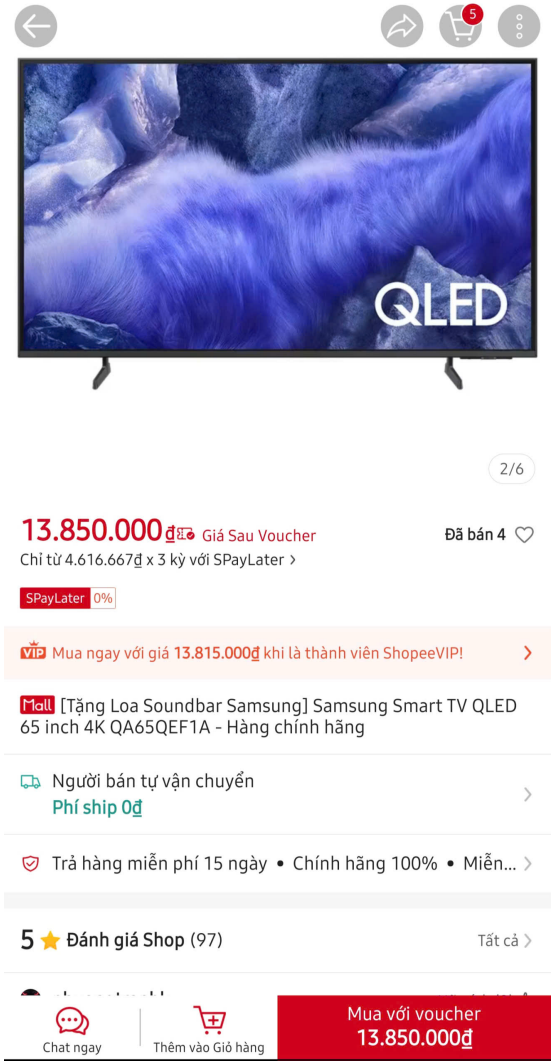
Presentation — UI, product listing, cart

Business Logic — pricing, vouchers, orders

Data Access — queries, updates, writes

Database

Layered Pattern in the Real World — Shopee



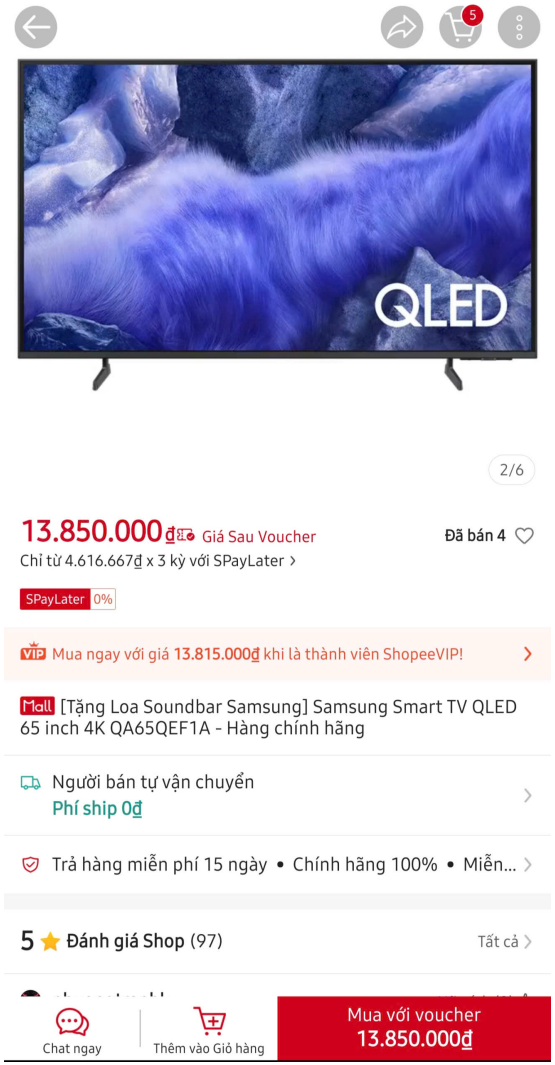
Presentation — UI, product listing, cart

Business Logic — pricing, vouchers, orders

Data Access — queries, updates, writes

Database — products, users, transactions

Layered Pattern in the Real World — Shopee



Presentation — UI, product listing, cart

Business Logic — pricing, vouchers, orders

Data Access — queries, updates, writes

Database — products, users, transactions

When you tap **"Buy Now"**:

Presentation tells Logic

Logic checks stock and voucher.

Data Access writes the order.

Database stores it.

The UI never touches the database directly — ever.

4. Software Architecture Process

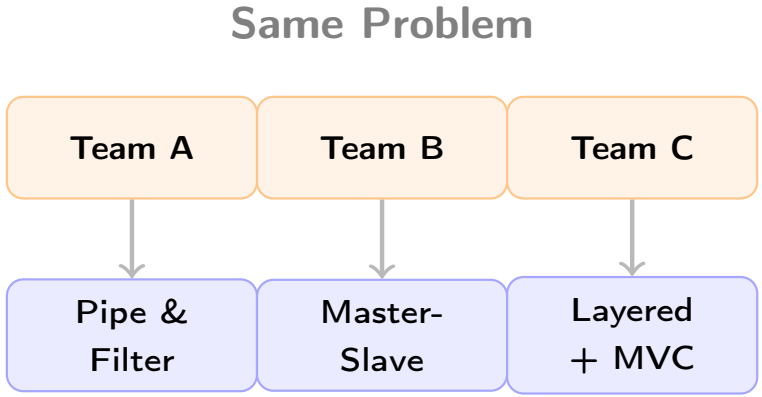
Software Architecture Process

Not a fixed recipe

- No strict step 1 → step 2 → step 3
- A **list of objectives** to accomplish
- Learn from past problems — design a unique solution
- Depth depends on project size

Two Goals of Architecture

- How the program **decomposes** internally
- How it **communicates** with users & other systems



3 completely different architectures — all valid

Subsystems vs Modules

Subsystem

Independent system with **standalone value**

Can be plugged into another program

Could be **sold** on its own

Module

Component **inside** a subsystem

Cannot function standalone

No independent value by itself

Breaking into subsystems ⇒ divide teams naturally

Subsystems vs Modules

Subsystem

Independent system with **standalone value**

Can be plugged into another program

Could be **sold** on its own

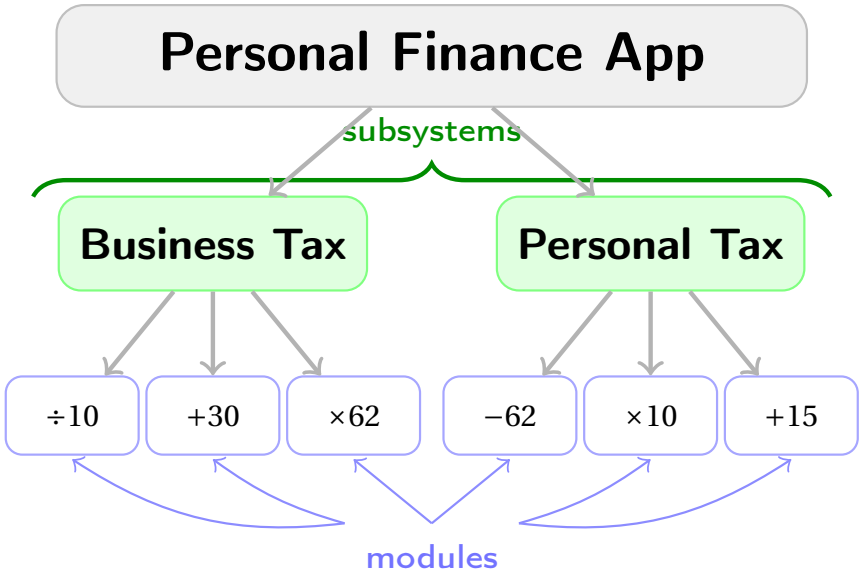
Module

Component **inside** a subsystem

Cannot function standalone

No independent value by itself

Breaking into subsystems ⇒ divide teams naturally



Subsystems vs Modules

Subsystem

Independent system with **standalone value**

Can be plugged into another program

Could be **sold** on its own

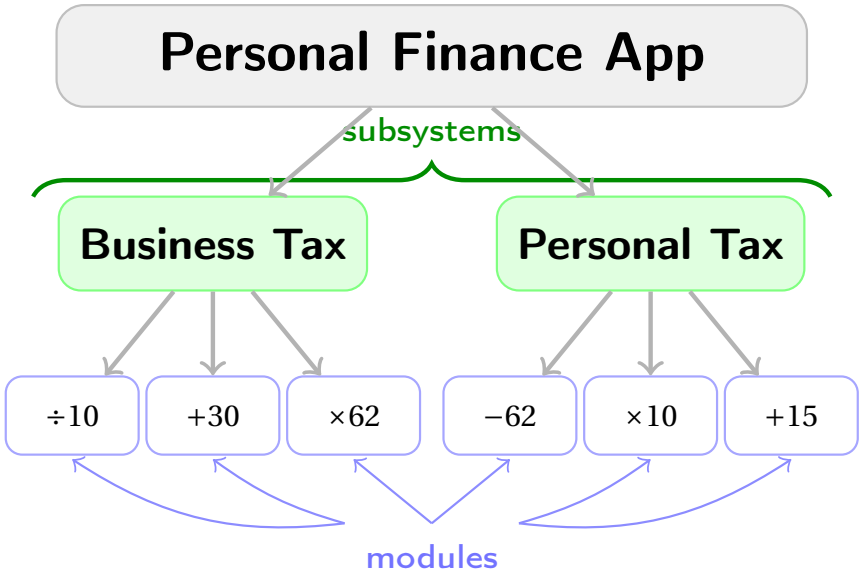
Module

Component **inside** a subsystem

Cannot function standalone

No independent value by itself

Breaking into subsystems ⇒ divide teams naturally



Input: income data

Subsystem: calculates tax — can be sold

Module: one math step — meaningless alone

App: combines subsystems into a full product

What to Keep in Mind

Quality Attribute	Key Question to Ask
Dependability	Will the system reliably do what it is supposed to do?
Effectiveness	Does this architecture actually solve our problem well?
Durability	Will it hold up over time and under more data or users?
Reproducibility	Can this solution be reused for the same type of problem again?
Vulnerability	How secure is it? Does it expose any known risks?

What to Keep in Mind

Quality Attribute	Key Question to Ask
Dependability	Will the system reliably do what it is supposed to do?
Effectiveness	Does this architecture actually solve our problem well?
Durability	Will it hold up over time and under more data or users?
Reproducibility	Can this solution be reused for the same type of problem again?
Vulnerability	How secure is it? Does it expose any known risks?

You don't need to answer all of them deeply —
but always make sure you have **thought** about each one